

PnD Manifest Correction

IO Resolution & Dependency Refinement Design

Version: 1.0

Date: February 2026

Status: Design Specification

Table of Contents

- [1. Problem Statement](#)
- [2. Proposed Solution Architecture](#)
- [3. Stage 2b: IO Resolution](#)
- [4. Stage 2c: Dependency Refinement](#)
- [5. Updated Workflow Manifest Schema](#)
- [6. Implementation Details](#)
- [7. System Prompt Improvements](#)

1. Problem Statement

1.1 Current Issues

Issue 1: Guessed Input/Output Schemas

```
// Current (WRONG) - LLM guesses schemas
{
  "id": "task_2",
  "input_schema": {
    "origin_airport": "LHR", // ✗ Guessed values
    "destination_airport": "NRT"
  },
  "output_schema": {
    "flights": [{ "flight_number": "AA1234" }] // ✗ Guessed structure
  }
}
```

Issue 2: Missing Capability References

```
// Current (WRONG) - No capability mapping
{
```

```

    "id": "task_2",
    "agent_id": "did:metaorcha:agent:skylink-flight-search",
    "description": "Find flights..."
    // ❌ Which MCP tool/function to call?
    // ❌ What are the actual input/output schemas?
  }

```

Issue 3: Over-Parallelized Dependencies

```

// Current (WRONG) - Everything depends on everything
"edges": [
  {"source": "task_1", "target": "task_2"},
  {"source": "task_1", "target": "task_3"},
  {"source": "task_1", "target": "task_4"},
  {"source": "task_2", "target": "task_3"}, // ✓ Valid
  {"source": "task_2", "target": "task_4"}, // ❌ Unnecessary
  {"source": "task_3", "target": "task_4"}, // ❌ Unnecessary
  // ... 10 more unnecessary edges
]

```

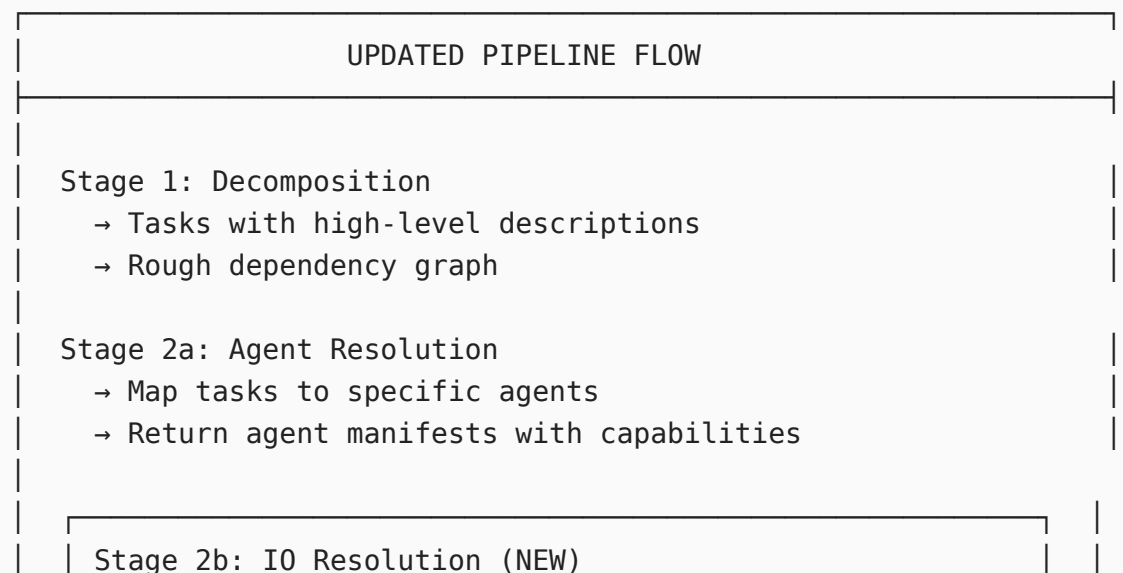
Issue 4: Unresolved Required Inputs

```

// User query: "Book a flight to Tokyo"
// Agent requires: origin, destination, departure_date, passengers,
// cabin_class
//
// Resolved from query: destination = "Tokyo"
// Missing: origin, departure_date, passengers, cabin_class
//
// ❌ No mechanism to collect these at runtime

```

1.2 Required Solution



- Map to exact MCP tools/functions
- Extract input values from user query
- Identify missing required inputs
- Create human-in-the-loop node if needed
- Set proper input/output schemas from manifest

Stage 2c: Dependency Refinement (NEW)

- Analyze data flow between tasks
- Remove transitive edges
- Keep only direct data dependencies
- Preserve sequential ordering where needed

Stage 3: Validation

- Verify IO compatibility
- Validate dependency graph

2. Proposed Solution Architecture

2.1 New Pipeline Stages

```
# src/planning/pipeline.py

class OptimizedPlanningPipeline:
    """Updated pipeline with IO resolution and dependency refinement"""

    async def plan(self, query: str) -> WorkflowManifest:
        """
        Complete planning pipeline:
        1. Decomposition → rough DAG
        2a. Agent Resolution → matched agents
        2b. IO Resolution → exact capabilities + schemas [NEW]
        2c. Dependency Refinement → clean DAG [NEW]
        2. Validation → final manifest
        """

        # Stage 1: Decomposition
        decomposition = await self.decomposer.decompose(query)

        # Stage 2a: Agent Resolution
        resolved_tasks = await self.resolver.resolve_agents(
            decomposition.tasks
```

```

    )

    # Stage 2b: IO Resolution (NEW)
    io_resolved_tasks, hitl_node = await self.io_resolver.resolve_io(
        tasks=resolved_tasks,
        user_query=query
    )

    # Stage 2c: Dependency Refinement (NEW)
    refined_dag = await self.dependency_refiner.refine_dependencies(
        tasks=io_resolved_tasks,
        initial_edges=decomposition.edges
    )

    # Stage 3: Validation
    manifest = await self.validator.validate(
        nodes=refined_dag.nodes,
        edges=refined_dag.edges,
        hitl_node=hitl_node
    )

    return manifest

```

3. Stage 2b: IO Resolution

3.1 Purpose

Map resolved agents to **exact capabilities** and extract/validate input parameters.

3.2 IO Resolution Process

```

# src/planning/resolution/io_resolver.py

from typing import List, Dict, Tuple, Optional
from pydantic import BaseModel

class IOResolutionResult(BaseModel):
    """Result of IO resolution"""
    capability_id: str
    capability_type: str # TOOL, RESOURCE, PROMPT
    capability_name: str
    input_schema: Dict
    filled_inputs: Dict # Values extracted from query OR data references
    missing_required_inputs: List[str]
    output_schema: Dict # Full schema with type annotations (not empty)
    data_dependencies: List[str] = [] # Task IDs this task depends on

```

```
field_mappings: List[Dict] = [] # Semantic field mappings with
confidence
```

```
class IOResolver:
```

```
    """
```

```
    Resolves exact capabilities and input/output schemas.
```

```
    Process:
```

1. Select best capability from agent manifest
2. Extract input values from user query
3. Identify missing required inputs
4. Create human-in-the-loop node if needed

```
    """
```

```
    def __init__(self, llm_provider):
```

```
        self.llm = llm_provider
```

```
    async def resolve_io(
```

```
        self,
```

```
        tasks: List[Dict],
```

```
        user_query: str
```

```
) -> Tuple[List[Dict], Optional[Dict]]:
```

```
    """
```

```
    Main entry point for IO resolution.
```

```
    Returns:
```

```
        (resolved_tasks, human_in_loop_node)
```

```
    """
```

```
        resolved_tasks = []
```

```
        all_missing_inputs = {} # task_id -> missing inputs
```

```
        for task in tasks:
```

```
            # Get agent manifest
```

```
            agent_manifest = await
```

```
self._fetch_agent_manifest(task['agent_id'])
```

```
            # Select best capability
```

```
            capability = await self._select_capability(
```

```
                task=task,
```

```
                agent_manifest=agent_manifest
```

```
            )
```

```
            # Extract input values from query
```

```
            io_result = await self._resolve_inputs(
```

```
                task=task,
```

```
                capability=capability,
```

```
                user_query=user_query
```

```
            )
```

```

        # Track missing inputs
        if io_result.missing_required_inputs:
            all_missing_inputs[task['id']] =
io_result.missing_required_inputs

        # Update task with resolved IO
        resolved_task = self._create_resolved_task(task, io_result)
        resolved_tasks.append(resolved_task)

    # Create human-in-the-loop node if needed
    hitl_node = None
    if all_missing_inputs:
        hitl_node = self._create_hitl_node(all_missing_inputs)

    return resolved_tasks, hitl_node

async def _select_capability(
    self,
    task: Dict,
    agent_manifest: Dict
) -> Dict:
    """
    Select the best capability from agent manifest.

    Strategy:
    1. Semantic matching between task description and capability
descriptions
    2. Use cross-encoder to rank capabilities
    3. Return highest-scoring capability
    """

    capabilities = agent_manifest.get('capabilities', {})
    all_caps = []

    # Flatten capabilities (tools, resources, prompts)
    for cap_type in ['tools', 'resources', 'prompts']:
        for cap in capabilities.get(cap_type, []):
            all_caps.append({
                'type': cap_type.rstrip('s').upper(), # TOOL,
RESOURCE, PROMPT
                'capability_id': cap.get('name'),
                'name': cap.get('name'),
                'description': cap.get('description', ''),
                'input_schema': cap.get('inputSchema', {}),
                'output_schema': cap.get('outputSchema', {})
            })

    if not all_caps:

```

```

        raise ValueError(f"Agent {agent_manifest['id']} has no
capabilities")

    # Semantic matching
    task_description = task.get('description', '')

    # Use LLM for capability selection
    selection_prompt = f"""Select the best capability for this task.

```

```

TASK: {task_description}

```

```

AVAILABLE CAPABILITIES:

```

```

{json.dumps(all_caps, indent=2)}

```

```

Return the index (0-based) of the best matching capability.

```

```

Output only the integer index, nothing else.

```

```

"""

```

```

    response = await self.llm.complete(
        model="gemini-1.5-flash",
        messages=[{"role": "user", "content": selection_prompt}],
        temperature=0
    )

```

```

    selected_idx = int(response.strip())
    return all_caps[selected_idx]

```

```

async def _resolve_inputs(
    self,
    task: Dict,
    capability: Dict,
    user_query: str
) -> IOResolutionResult:

```

```

    """

```

```

    Extract input values from user query and identify missing inputs.

```

```

    Uses LLM to extract structured data from natural language.

```

```

    """

```

```

    input_schema = capability['input_schema']
    properties = input_schema.get('properties', {})
    required_fields = input_schema.get('required', [])

```

```

    # Extract values using LLM

```

```

    extraction_prompt = f"""Extract input values from the user query
for this capability.

```

```

USER QUERY: {user_query}

```

```
TASK CONTEXT: {task.get('description', '')}
```

INPUT SCHEMA:

```
{json.dumps(input_schema, indent=2)}
```

Extract values for each property. If a value cannot be determined from the query, set it to null.

Output JSON format:

```
{  
  "extracted_inputs": {  
    "property_name": "extracted_value or null"  
  }  
}
```

```
response = await self.llm.complete(  
    model="gemini-1.5-flash",  
    messages=[{"role": "user", "content": extraction_prompt}],  
    response_format={"type": "json_object"},  
    temperature=0  
)
```

```
extraction = json.loads(response)  
extracted_inputs = extraction.get('extracted_inputs', {})
```

```
# Identify missing required inputs
```

```
filled_inputs = {}
```

```
missing_required = []
```

```
for field_name, field_schema in properties.items():  
    extracted_value = extracted_inputs.get(field_name)
```

```
    if extracted_value is not None:
```

```
        # Value was extracted
```

```
        filled_inputs[field_name] = extracted_value
```

```
    elif field_name in required_fields:
```

```
        # Required but missing
```

```
        missing_required.append(field_name)
```

```
    elif 'default' in field_schema:
```

```
        # Use default value
```

```
        filled_inputs[field_name] = field_schema['default']
```

```
return IOResolutionResult(  
    capability_id=capability['capability_id'],  
    capability_type=capability['type'],  
    capability_name=capability['name'],  
    input_schema=input_schema,  
    filled_inputs=filled_inputs,
```

```

        missing_required_inputs=missing_required,
        output_schema=self._create_empty_output_schema(
            capability['output_schema']
        )
    )

def _preserve_output_schema(self, output_schema: Dict) -> Dict:
    """
    Preserve full output schema with type annotations for runtime
    validation.

    Example:
    Input: {
        "type": "object",
        "properties": {
            "price": {"type": "number", "description": "Price in USD"},
            "currency": {"type": "string", "enum": ["USD", "GBP", "JPY"]}
        }
    }
    Output: Same as input (full schema preserved)

    Runtime will use this schema to:
    1. Validate actual output values
    2. Understand data types for downstream tasks
    3. Provide type hints for development
    """

    # Return the complete schema as-is
    return output_schema

def _create_resolved_task(
    self,
    original_task: Dict,
    io_result: IOResolutionResult
) -> Dict:
    """Create task with resolved IO"""

    return {
        "id": original_task['id'],
        "type": "standard",
        "agent_id": original_task['agent_id'],
        "capability": {
            "capability_id": io_result.capability_id,
            "type": io_result.capability_type,
            "name": io_result.capability_name,
            "input_schema": io_result.input_schema,
            "output_schema": io_result.output_schema
        },
        "task": {

```

```

        "description": original_task.get('description', ''),
        "inputs": io_result.filled_inputs
    },
    "depends_on": original_task.get('depends_on', []),
    "unresolved_inputs": io_result.missing_required_inputs
}

def _create_hitl_node(self, all_missing_inputs: Dict[str, List[str]])
-> Dict:
    """
    Create human-in-the-loop node to collect missing inputs.

    This node runs FIRST and collects all missing required inputs.
    """

    # Flatten all missing inputs with task references
    input_requests = []

    for task_id, missing_fields in all_missing_inputs.items():
        for field in missing_fields:
            input_requests.append({
                "target_task": task_id,
                "field_name": field,
                "prompt": f"Please provide {field} for {task_id}"
            })

    return {
        "id": "hitl_input_collection",
        "type": "system_tool",
        "tool": "human_input",
        "inputs": {
            "requests": input_requests
        },
        "depends_on": [],
        "outputs": {
            "collected_inputs": {} # Will be filled at runtime
        }
    }

```

3.3 Data Dependencies Between Tasks

```

# src/planning/resolution/io_resolver.py (continued)

class IOResolver:
    """IO Resolver with data dependency tracking"""

    async def _resolve_inputs(
        self,
        task: Dict,

```



```

        'source_field': output_field,
        'target_field': field_name,
        'confidence': providing_info['confidence'],
        'reasoning': providing_info['reasoning']
    })

    continue

# Strategy 3: Check for default value
if 'default' in field_schema:
    filled_inputs[field_name] = field_schema['default']
    continue

# Strategy 4: Mark as missing if required
if field_name in input_schema.get('required', []):
    missing_required.append(field_name)

return IOResolutionResult(
    capability_id=capability['capability_id'],
    capability_type=capability['type'],
    capability_name=capability['name'],
    input_schema=input_schema,
    filled_inputs=filled_inputs,
    missing_required_inputs=missing_required,
    output_schema=capability['output_schema'], # Full schema with
types
    data_dependencies=data_dependencies,
    field_mappings=field_mappings # NEW: Track semantic mappings
)

async def _find_providing_task(
    self,
    field_name: str,
    field_schema: Dict,
    previous_tasks: List[Dict]
) -> Optional[Dict]:
    """
    Find which previous task can provide this input using SEMANTIC
matching.

    Problem:
    - Tasks use different field names for same data
    - Example: "cost" vs "price" vs "amount"
    - Example: "booking_id" vs "confirmation_number" vs
"reservation_id"

    Solution:
    - Use LLM to semantically match fields
    - Consider field descriptions, not just names

```

- Match by data type and semantic meaning

Strategy:

1. Build candidate list from previous task outputs
2. Use LLM to match based on semantic similarity
3. Return task that produces the semantically equivalent output

```
if not previous_tasks:
    return None

# Build list of candidate fields from previous tasks
candidates = []

for task in reversed(previous_tasks): # Most recent first
    if 'capability' not in task:
        continue

    output_schema = task['capability'].get('output_schema', {})
    output_props = output_schema.get('properties', {})

    for output_field_name, output_field_schema in
output_props.items():
        candidates.append({
            'task_id': task['id'],
            'task_description': task.get('task',
{}).get('description', ''),
            'field_name': output_field_name,
            'field_type': output_field_schema.get('type'),
            'field_description':
output_field_schema.get('description', ''),
            'enum_values': output_field_schema.get('enum', [])
        })

    if not candidates:
        return None

# Use LLM for semantic matching
matching_prompt = f"""Match the required input field to the best
available output field.

REQUIRED INPUT FIELD:
Name: {field_name}
Type: {field_schema.get('type')}
Description: {field_schema.get('description', 'N/A')}

AVAILABLE OUTPUT FIELDS FROM PREVIOUS TASKS:
{json.dumps(candidates, indent=2)}
```

INSTRUCTIONS:

1. Find the output field that SEMANTICALLY matches the required input
2. Consider field descriptions and context, not just names
3. Ensure type compatibility
4. If no good match exists, return null

Examples of semantic matches:

- "cost" matches "price" or "total_amount"
- "booking_id" matches "confirmation_number" or "reservation_id"
- "destination_city" matches "city" or "location"
- "base_currency" matches "from_currency" or "source_currency"

Output JSON format:

```
{{
  "matched_task_id": "task_2" or null,
  "matched_field_name": "price" or null,
  "confidence": 0.95,
  "reasoning": "The 'price' field from hotel booking semantically matches
'cost' as both represent monetary amounts"
}}
```

```
response = await self.llm.complete(
    model="gemini-1.5-flash",
    messages=[{"role": "user", "content": matching_prompt}],
    response_format={"type": "json_object"},
    temperature=0
)

match_result = json.loads(response)

# Only use match if confidence is high enough
if match_result.get('confidence', 0) >= 0.7 and
match_result.get('matched_task_id'):
    # Find and return the matched task
    for task in previous_tasks:
        if task['id'] == match_result['matched_task_id']:
            # Store the field mapping for later reference
            return {
                'task': task,
                'output_field':
match_result['matched_field_name'],
                'confidence': match_result['confidence'],
                'reasoning': match_result.get('reasoning', '')
            }

    return None
```

4.3 Semantic Field Mapping

4.3.1 The Challenge

Different agents use different field names for semantically equivalent data:

```
Hotel Agent outputs:    → "total_cost", "currency"
Currency Agent expects: → "amount", "from_currency"

Flight Agent outputs:   → "arrival_time"
Hotel Agent expects:    → "check_in_date"

Payment outputs:        → "transaction_id"
Booking expects:        → "confirmation_number"
```

4.3.2 LLM-Based Semantic Matching

Instead of exact field name matching, we use LLM to understand semantic equivalence:

```
# WRONG: Exact name matching
if output_field_name == input_field_name: # ❌ Fails for "cost" vs
    "amount"
    create_data_reference()

# RIGHT: Semantic matching
match = await llm.match_fields(
    required_field="amount",
    available_fields=["total_cost", "price", "booking_fee"]
)
# Returns: "total_cost" with confidence 0.95
```

4.3.3 Confidence Scoring

Each field mapping includes confidence score and reasoning:

```
{
  "field_mappings": [
    {
      "source_task": "task_3",
      "source_field": "total_cost",
      "target_field": "amount",
      "confidence": 0.95,
      "reasoning": "Hotel total_cost semantically matches currency
conversion amount"
    },
    {
      "source_task": "task_2",
```

```

    "source_field": "flights[0].arrival_time",
    "target_field": "check_in_date",
    "confidence": 0.92,
    "reasoning": "Flight arrival time semantically matches hotel check-
in date"
  }
]
}

```

4.3.4 Array Access & Nested Fields

The system handles complex data structures:

```

{
  "inputs": {
    "check_in_date": "$tasks.task_2.output.flights[0].arrival_time",
    "price": "$tasks.task_3.output.booking_details.cost_breakdown.total"
  }
}

```

Runtime will:

1. Execute task_2 (flight search)
2. Extract `flights[0].arrival_time` from results
3. Pass to task_3 as `check_in_date`

4.3.5 Type Validation

Full output schemas enable runtime type checking:

```

# Output schema from flight search
output_schema = {
  "properties": {
    "flights": {
      "type": "array",
      "items": {
        "properties": {
          "arrival_time": {"type": "string", "format": "date-time"}
        }
      }
    }
  }
}

# Runtime validates:
# - flights is actually an array

```

```
# - flights[0].arrival_time is a valid date-time string
# - Can be used for check_in_date (also date type)
```

4. Stage 2c: Dependency Refinement

4.1 Purpose

Clean up the dependency graph by removing unnecessary edges and preserving only **direct data dependencies**.

4.2 Dependency Refinement Algorithm

```
# src/planning/resolution/dependency_refiner.py

from typing import List, Dict, Set, Tuple
import networkx as nx

class DependencyRefiner:
    """
    Refines dependency graph to remove unnecessary edges.

    Principles:
    1. Keep edges where task B depends EXCLUSIVELY on task A's output
    2. Remove transitive edges (if A→B→C exists, remove A→C)
    3. Preserve sequential ordering from decomposition
    4. Enable parallelization where truly independent
    """

    async def refine_dependencies(
        self,
        tasks: List[Dict],
        initial_edges: List[Dict]
    ) -> Dict:
        """
        Main entry point for dependency refinement.

        Returns:
            {"nodes": [...], "edges": [...]}
        """

        # Build task lookup
        task_map = {t['id']: t for t in tasks}

        # Step 1: Build data dependency graph
        data_dependencies = self._extract_data_dependencies(tasks)
```

```

# Step 2: Build sequential dependency graph
sequential_dependencies = self._extract_sequential_dependencies(
    tasks,
    initial_edges
)

# Step 3: Merge and remove transitive edges
refined_edges = self._merge_and_reduce(
    data_dependencies,
    sequential_dependencies,
    task_map
)

return {
    "nodes": tasks,
    "edges": refined_edges
}

def _extract_data_dependencies(self, tasks: List[Dict]) -> Dict[str,
Set[str]]:
    """
    Extract data dependencies by analyzing input references.

    Returns: {task_id: {set of task_ids it depends on}}
    """

    dependencies = {}

    for task in tasks:
        task_deps = set()

        # Check task inputs for data references
        task_inputs = task.get('task', {}).get('inputs', {})

        for input_value in task_inputs.values():
            if isinstance(input_value, str) and
input_value.startswith('$tasks.'):
                # Parse reference: $tasks.task_2.output.price
                ref_parts = input_value.split('.')
                if len(ref_parts) >= 2:
                    referenced_task_id = ref_parts[1]
                    task_deps.add(referenced_task_id)

        # Also check explicit data_dependencies from IO resolution
        if 'data_dependencies' in task:
            task_deps.update(task['data_dependencies'])

    dependencies[task['id']] = task_deps

```

```

        return dependencies

def _extract_sequential_dependencies(
    self,
    tasks: List[Dict],
    initial_edges: List[Dict]
) -> Dict[str, Set[str]]:
    """
    Extract sequential dependencies from initial decomposition.

    Only preserve if there's a semantic reason (not just ordering).
    """

    # Use LLM to determine if sequential dependency is necessary
    # For now, use initial edges but filter out obvious parallels

    dependencies = {}

    for edge in initial_edges:
        source = edge.get('source') or edge.get('from')
        target = edge.get('target') or edge.get('to')

        if target not in dependencies:
            dependencies[target] = set()

        dependencies[target].add(source)

    return dependencies

def _merge_and_reduce(
    self,
    data_deps: Dict[str, Set[str]],
    seq_deps: Dict[str, Set[str]],
    task_map: Dict[str, Dict]
) -> List[Dict]:
    """
    Merge data and sequential dependencies, then remove transitive
edges.
    """

    # Merge dependencies
    all_deps = {}
    for task_id in task_map.keys():
        merged = set()
        merged.update(data_deps.get(task_id, set()))
        merged.update(seq_deps.get(task_id, set()))
        all_deps[task_id] = merged

    # Build directed graph

```

```

G = nx.DiGraph()
for task_id in task_map.keys():
    G.add_node(task_id)

for task_id, deps in all_deps.items():
    for dep in deps:
        G.add_edge(dep, task_id)

# Remove transitive edges
# If A→B and B→C exist, remove A→C
reduced_edges = self._transitive_reduction(G)

# Convert back to edge list
edges = []
for source, target in reduced_edges:
    edges.append({
        "from": source,
        "to": target,
        "condition": None
    })

return edges

def _transitive_reduction(self, G: nx.DiGraph) -> List[Tuple[str,
str]]:
    """
    Remove transitive edges from DAG.

    Uses NetworkX's transitive_reduction algorithm.
    """

    reduced_graph = nx.transitive_reduction(G)
    return list(reduced_graph.edges())

```

4.3 Example: Before & After Refinement

Before (Over-Parallelized):

```

{
  "edges": [
    {"from": "task_1", "to": "task_2"}, // ✓ Keep
    {"from": "task_1", "to": "task_3"}, // ✗ Remove (transitive via
task_2)
    {"from": "task_1", "to": "task_4"}, // ✗ Remove (transitive)
    {"from": "task_2", "to": "task_3"}, // ✓ Keep (data dependency)
    {"from": "task_2", "to": "task_4"}, // ✗ Remove (transitive via
task_3)
    {"from": "task_3", "to": "task_4"} // ✓ Keep (data dependency)
  ]
}

```

```
]
}
```

After (Refined):

```
{
  "edges": [
    {"from": "task_1", "to": "task_2"}, // Direct dependency
    {"from": "task_2", "to": "task_3"}, // Direct data dependency
    {"from": "task_3", "to": "task_4"}  // Direct data dependency
  ]
}
```

Execution:

- task_1 runs first
 - task_2 runs after task_1 completes
 - task_3 runs after task_2 completes
 - task_4 runs after task_3 completes
-

5. Updated Workflow Manifest Schema

5.1 Final StandardNode Schema

```
# src/schemas/workflow_manifest.py

class CapabilityReference(BaseModel):
    """Reference to exact agent capability"""
    capability_id: str # e.g., "search_flights"
    type: Literal["TOOL", "RESOURCE", "PROMPT"]
    name: str
    input_schema: Dict # Full schema from agent manifest
    output_schema: Dict # Empty structure (keys only)

class TaskDescriptor(BaseModel):
    """Task with resolved inputs"""
    description: str
    inputs: Dict[str, Any] # Can contain values OR data references
    # Data references: "$tasks.task_2.output.price"

class StandardNode(BaseModel):
    """Agent task node with resolved IO"""
    id: str
    type: Literal["standard"] = "standard"
    agent_id: str # DID
```

```

capability: CapabilityReference # NEW: Exact capability mapping
task: TaskDescriptor
depends_on: List[str]
unresolved_inputs: List[str] = [] # Fields needing human input
timeout_seconds: int = 30
retry_policy: Optional[Dict] = None

```

```

class HumanInputNode(BaseModel):
    """Human-in-the-loop node for collecting missing inputs"""
    id: str = "hitl_input_collection"
    type: Literal["system_tool"] = "system_tool"
    tool: Literal["human_input"] = "human_input"
    inputs: Dict[str, List[Dict]] # List of input requests
    depends_on: List[str] = []
    outputs: Dict = {"collected_inputs": {}}

```

5.2 Complete Example

```

{
  "workflow": {
    "id": "workflow_123",
    "version": "4.0",
    "nodes": [
      {
        "id": "hitl_input_collection",
        "type": "system_tool",
        "tool": "human_input",
        "inputs": {
          "requests": [
            {
              "target_task": "task_2",
              "field_name": "departure_date",
              "field_type": "string",
              "field_format": "date",
              "prompt": "When would you like to depart? (YYYY-MM-DD)",
              "description": "Departure date for flight search"
            },
            {
              "target_task": "task_2",
              "field_name": "passengers",
              "field_type": "integer",
              "prompt": "How many passengers?",
              "description": "Number of passengers (1-9)",
              "validation": {"min": 1, "max": 9}
            }
          ]
        }
      },
      {
        "target_task": "task_2",
        "field_name": "passengers",
        "field_type": "integer",
        "prompt": "How many passengers?",
        "description": "Number of passengers (1-9)",
        "validation": {"min": 1, "max": 9}
      }
    ],
    "depends_on": [],
    "outputs": {

```

```

        "collected_inputs": {}
    }
},
{
    "id": "task_1",
    "type": "standard",
    "agent_id": "did:metaorcha:agent:wandercraft",
    "capability": {
        "capability_id": "generate_itinerary",
        "type": "T00L",
        "name": "Generate Itinerary",
        "input_schema": {
            "type": "object",
            "properties": {
                "destination": {"type": "string"},
                "duration_days": {"type": "integer"}
            },
            "required": ["destination", "duration_days"]
        },
        "output_schema": {
            "type": "object",
            "properties": {
                "destination": {
                    "type": "string",
                    "description": "Destination city"
                },
                "duration_days": {
                    "type": "integer",
                    "description": "Trip duration in days"
                },
                "daily_activities": {
                    "type": "array",
                    "items": {
                        "type": "object",
                        "properties": {
                            "day": {"type": "integer"},
                            "activities": {"type": "array"}
                        }
                    }
                }
            }
        }
    },
    "task": {
        "description": "Plan 7-day trip to Tokyo",
        "inputs": {
            "destination": "Tokyo",
            "duration_days": 7
        }
    }
}

```

```

    },
    "depends_on": ["hitl_input_collection"],
    "unresolved_inputs": []
  },
  {
    "id": "task_2",
    "type": "standard",
    "agent_id": "did:metaorcha:agent:skylink",
    "capability": {
      "capability_id": "search_flights",
      "type": "T00L",
      "name": "Search Flights",
      "input_schema": {
        "type": "object",
        "properties": {
          "origin": {"type": "string"},
          "destination": {"type": "string"},
          "departure_date": {"type": "string", "format": "date"},
          "passengers": {"type": "integer", "default": 1}
        },
        "required": ["origin", "destination", "departure_date"]
      },
      "output_schema": {
        "type": "object",
        "properties": {
          "flights": {
            "type": "array",
            "items": {
              "type": "object",
              "properties": {
                "flight_id": {"type": "string"},
                "price": {"type": "number"},
                "currency": {"type": "string"},
                "departure_time": {"type": "string", "format": "date-time"},
                "arrival_time": {"type": "string", "format": "date-time"}
              }
            }
          },
          "total_results": {"type": "integer"},
          "cheapest_price": {"type": "number"},
          "currency": {"type": "string"}
        }
      }
    },
    "task": {
      "description": "Search flights LHR to NRT",
      "inputs": {

```

```

        "origin": "LHR",
        "destination": "$tasks.task_1.output.destination",
        "departure_date":
"$hitl.collected_inputs.task_2.departure_date",
        "passengers": "$hitl.collected_inputs.task_2.passengers"
    }
},
"depends_on": ["task_1", "hitl_input_collection"],
"unresolved_inputs": [],
"field_mappings": [
    {
        "source_task": "task_1",
        "source_field": "destination",
        "target_field": "destination",
        "confidence": 0.99,
        "reasoning": "Exact field name match for destination city"
    }
]
},
{
    "id": "task_3",
    "type": "standard",
    "agent_id": "did:metaorcha:agent:staywise",
    "capability": {
        "capability_id": "book_hotel",
        "type": "T00L",
        "name": "Book Hotel",
        "input_schema": {
            "type": "object",
            "properties": {
                "location": {"type": "string"},
                "check_in_date": {"type": "string", "format": "date"},
                "num_nights": {"type": "integer"},
                "max_budget_per_night": {"type": "number"}
            },
            "required": ["location", "check_in_date", "num_nights"]
        },
        "output_schema": {
            "type": "object",
            "properties": {
                "booking_id": {"type": "string"},
                "hotel_name": {"type": "string"},
                "cost_per_night": {"type": "number"},
                "total_cost": {"type": "number"},
                "currency": {"type": "string"}
            }
        }
    }
},
"task": {

```

```

        "description": "Book hotel in Shibuya",
        "inputs": {
            "location": "Shibuya, Tokyo",
            "check_in_date":
"$tasks.task_2.output.flights[0].arrival_time",
            "num_nights": 5,
            "max_budget_per_night": 150
        }
    },
    "depends_on": ["task_2"],
    "unresolved_inputs": [],
    "field_mappings": [
        {
            "source_task": "task_2",
            "source_field": "flights[0].arrival_time",
            "target_field": "check_in_date",
            "confidence": 0.92,
            "reasoning": "Flight arrival time semantically matches hotel
check-in date"
        }
    ]
},
{
    "id": "task_4",
    "type": "standard",
    "agent_id": "did:metaorcha:agent:forex",
    "capability": {
        "capability_id": "convert_currency",
        "type": "T00L",
        "name": "Convert Currency",
        "input_schema": {
            "type": "object",
            "properties": {
                "amount": {"type": "number"},
                "from_currency": {"type": "string"},
                "to_currency": {"type": "string"}
            },
            "required": ["amount", "from_currency", "to_currency"]
        },
        "output_schema": {
            "type": "object",
            "properties": {
                "converted_amount": {"type": "number"},
                "exchange_rate": {"type": "number"},
                "source_currency": {"type": "string"},
                "target_currency": {"type": "string"}
            }
        }
    }
},

```

```

    "task": {
      "description": "Convert hotel cost to JPY",
      "inputs": {
        "amount": "$tasks.task_3.output.total_cost",
        "from_currency": "$tasks.task_3.output.currency",
        "to_currency": "JPY"
      }
    },
    "depends_on": ["task_3"],
    "unresolved_inputs": [],
    "field_mappings": [
      {
        "source_task": "task_3",
        "source_field": "total_cost",
        "target_field": "amount",
        "confidence": 0.95,
        "reasoning": "Hotel total_cost semantically matches currency
conversion amount"
      },
      {
        "source_task": "task_3",
        "source_field": "currency",
        "target_field": "from_currency",
        "confidence": 0.98,
        "reasoning": "Hotel currency matches source currency for
conversion"
      }
    ]
  },
  "edges": [
    {"from": "hitl_input_collection", "to": "task_1"},
    {"from": "hitl_input_collection", "to": "task_2"},
    {"from": "task_1", "to": "task_2"},
    {"from": "task_2", "to": "task_3"},
    {"from": "task_3", "to": "task_4"}
  ]
}

```

6. Implementation Details

6.1 Updated Planning Pipeline

```
# src/planning/pipeline.py
```

```

class OptimizedPlanningPipeline:
    """Complete pipeline with IO resolution and dependency refinement"""

    def __init__(self, db_pool, llm_provider):
        self.decomposer = SinglePassDecomposer(llm_provider)
        self.resolver = HybridSearchPipeline(db_pool, llm_provider)
        self.io_resolver = IOResolver(llm_provider) # NEW
        self.dependency_refiner = DependencyRefiner(llm_provider) # NEW
        self.validator = TieredValidator(llm_provider)

    async def plan(self, user_query: str) -> WorkflowManifest:
        """
        Complete planning with 5 stages:
        1. Decomposition
        2a. Agent Resolution
        2b. IO Resolution (NEW)
        2c. Dependency Refinement (NEW)
        2. Validation
        """

        logger.info(f"Planning workflow for query: {user_query}")

        # Stage 1: Decomposition
        logger.info("Stage 1: Decomposition")
        decomposition = await self.decomposer.decompose(user_query)

        # Stage 2a: Agent Resolution
        logger.info("Stage 2a: Agent Resolution")
        resolved_tasks = []

        for task in decomposition.tasks:
            if task['type'] != 'agent_task':
                continue

            # Search for agents
            agent_candidates = await self.resolver.search(
                query=task['description'],
                filters=task.get('required_capabilities', {})
            )

            if not agent_candidates:
                raise NoAgentsFoundError(f"No agents found for: {task['description']}")

            # Use top agent
            task['agent_id'] = agent_candidates[0]['id']
            task['agent_manifest'] = agent_candidates[0]['manifest']
            resolved_tasks.append(task)

```

```

# Stage 2b: IO Resolution
logger.info("Stage 2b: IO Resolution")
io_resolved_tasks, hitl_node = await self.io_resolver.resolve_io(
    tasks=resolved_tasks,
    user_query=user_query
)

# Stage 2c: Dependency Refinement
logger.info("Stage 2c: Dependency Refinement")
refined_dag = await self.dependency_refiner.refine_dependencies(
    tasks=io_resolved_tasks,
    initial_edges=decomposition.edges
)

# Add HITL node if needed
all_nodes = refined_dag['nodes']
all_edges = refined_dag['edges']

if hitl_node:
    all_nodes.insert(0, hitl_node)
    # Add edges from HITL to all tasks with unresolved inputs
    for task in all_nodes:
        if task.get('unresolved_inputs'):
            all_edges.insert(0, {
                "from": hitl_node['id'],
                "to": task['id']
            })

# Stage 3: Validation
logger.info("Stage 3: Validation")
validation_result = await self.validator.validate({
    "nodes": all_nodes,
    "edges": all_edges
})

if not validation_result.is_valid:
    raise ValidationFailedError(validation_result.errors)

# Create final manifest
manifest = WorkflowManifest(
    id=str(uuid.uuid4()),
    version="4.0",
    created_at=datetime.utcnow().isoformat(),
    nodes=all_nodes,
    edges=all_edges,
    validated=True,
    validation_tier=validation_result.tier,
    confidence_score=validation_result.confidence,
    planning_metrics={

```

```

        "total_time_ms": 0, # Track actual time
        "total_llm_calls": 0,
        "total_cost_usd": 0
    }
)

return manifest

```

7. System Prompt Improvements

7.1 Updated Decomposition Prompt

```
# src/planning/decomposition/prompts.py
```

```
DECOMPOSITION_SYSTEM_PROMPT = """You are an expert workflow architect.
```

```
CRITICAL RULES FOR DEPENDENCIES:
```

1. SEQUENTIAL BY DEFAULT

- Tasks are executed in the order you define them
- Only add edges where there's an EXPLICIT data dependency
- Do NOT create edges just because tasks happen sequentially

2. DATA DEPENDENCIES ONLY

- Add edge A→B ONLY if B needs output from A
- Example: "Book hotel after finding flights" → edge needed
- Example: "Search flights, then check weather" → NO edge if weather doesn't need flight data

3. PARALLEL EXECUTION

- Tasks WITHOUT data dependencies can run in parallel
- Runtime will execute independent tasks concurrently
- Don't artificially serialize independent tasks

```
EXAMPLES:
```

✗ WRONG (Over-parallelized):

Query: "Find flights, book hotel, check weather"

edges: [

```

    {from: "flights", to: "hotel"},
    {from: "flights", to: "weather"},
    {from: "hotel", to: "weather"}

```

]

→ NO! Hotel and weather don't need each other's data

✓ CORRECT (Data dependencies only):

```

Query: "Find flights to Tokyo, book hotel near airport"
edges: [
  {from: "flights", to: "hotel"} // Hotel needs airport from flights
]
→ YES! Hotel booking needs the destination airport

✓ CORRECT (Truly parallel):
Query: "Find flights, check weather, convert currency"
edges: [] // All independent, can run in parallel

✓ CORRECT (Sequential chain):
Query: "Get ETH price, if below $2000 then calculate RSI then execute trade"
edges: [
  {from: "get_price", to: "router_check"},
  {from: "router_check", to: "calculate_rsi", condition: "price < 2000"},
  {from: "calculate_rsi", to: "execute_trade"}
]

OUTPUT FORMAT:
{
  "tasks": [...],
  "edges": [
    {"from": "task_id", "to": "task_id", "condition": null}
  ]
}

Remember: Edges represent DATA FLOW, not just temporal ordering!
"""

```

7.2 IO Resolution Prompt

```

# src/planning/resolution/prompts.py

INPUT_EXTRACTION_PROMPT = """Extract input values from user query for this
capability.

USER QUERY: {user_query}

TASK CONTEXT: {task_description}

CAPABILITY INPUT SCHEMA:
{input_schema}

INSTRUCTIONS:
1. Extract values that are EXPLICITLY mentioned in the query
2. Do NOT guess or infer values
3. Use null for any value not found in the query
4. Match field names exactly as specified in schema

```

Examples:

Query: "Find flights from London to Tokyo on March 15th for 2 passengers"

Schema: {origin, destination, departure_date, passengers}

Output: {

```
"origin": "London",  
"destination": "Tokyo",  
"departure_date": "2026-03-15",  
"passengers": 2
```

}

Query: "Book a hotel in Shibuya"

Schema: {location, check_in_date, num_nights, budget}

Output: {

```
"location": "Shibuya",  
"check_in_date": null,    // Not mentioned  
"num_nights": null,      // Not mentioned  
"budget": null           // Not mentioned
```

}

Now extract for:

Query: {user_query}

Schema: {input_schema}

Output JSON:

```
{  
  "extracted_inputs": { ... }  
}  
"""
```

8. Testing Strategy

8.1 Test Cases for IO Resolution

```
# tests/unit/planning/resolution/test_io_resolver.py  
  
import pytest  
  
@pytest.mark.asyncio  
async def test_capability_selection():  
    """Test selecting best capability from agent manifest"""  
  
    agent_manifest = {  
        "capabilities": {  
            "tools": [  

```

```

        {
            "name": "search_flights",
            "description": "Search for available flights"
        },
        {
            "name": "book_flight",
            "description": "Book a specific flight"
        }
    ]
}

task = {
    "description": "Find flights from London to Tokyo"
}

resolver = IOResolver(mock_llm)
capability = await resolver._select_capability(task, agent_manifest)

assert capability['capability_id'] == "search_flights"

```

```

@pytest.mark.asyncio
async def test_input_extraction():
    """Test extracting input values from query"""

    query = "Find flights from LHR to NRT on 2026-03-15 for 2 passengers"

    input_schema = {
        "properties": {
            "origin": {"type": "string"},
            "destination": {"type": "string"},
            "departure_date": {"type": "string"},
            "passengers": {"type": "integer"}
        },
        "required": ["origin", "destination", "departure_date"]
    }

    resolver = IOResolver(mock_llm)
    result = await resolver._extract_from_query(query, input_schema)

    assert result['origin'] == "LHR"
    assert result['destination'] == "NRT"
    assert result['departure_date'] == "2026-03-15"
    assert result['passengers'] == 2

```

```

@pytest.mark.asyncio
async def test_missing_required_inputs():
    """Test identification of missing required inputs"""

```

```

# ... (test implementation)

@pytest.mark.asyncio
async def test_data_reference_creation():
    """Test creating data references to previous task outputs"""

    # ... (test implementation)

```

8.2 Test Cases for Dependency Refinement

```

# tests/unit/planning/resolution/test_dependency_refiner.py

import pytest

def test_transitive_reduction():
    """Test removing transitive edges"""

    tasks = [
        {"id": "task_1"},
        {"id": "task_2"},
        {"id": "task_3"}
    ]

    # A→B→C with redundant A→C
    edges = [
        {"from": "task_1", "to": "task_2"},
        {"from": "task_2", "to": "task_3"},
        {"from": "task_1", "to": "task_3"} # Redundant
    ]

    refiner = DependencyRefiner(None)
    refined = refiner.refine_dependencies(tasks, edges)

    # Should remove task_1 → task_3
    assert len(refined['edges']) == 2
    assert {"from": "task_1", "to": "task_3"} not in refined['edges']

def test_parallel_detection():
    """Test that truly parallel tasks have no edges"""

    tasks = [
        {"id": "task_1", "task": {"inputs": {}}},
        {"id": "task_2", "task": {"inputs": {}}},
        {"id": "task_3", "task": {"inputs": {}}}
    ]

    # No data dependencies
    edges = []

```

```
refiner = DependencyRefiner(None)
refined = refiner.refine_dependencies(tasks, edges)

assert len(refined['edges']) == 0
```

Summary of Changes

What Gets Fixed

1. No More Guessed Schemas

- LLM selects exact capability from agent manifest
- Input/output schemas come directly from agent capabilities
- Runtime knows exactly which MCP tool/function to call

2. Missing Inputs Handled

- System identifies required inputs not in user query
- Creates human-in-the-loop node to collect them
- Runtime can prompt user before execution

3. Clean Dependency Graph

- Removes transitive edges
- Only keeps direct data dependencies
- Enables proper parallel execution

4. Data Flow Tracking

- Explicit references between task outputs and inputs
- Runtime can resolve values at execution time
- Clear lineage for debugging

Performance Impact

- **2 Additional LLM Calls** per workflow:
 - IO Resolution: 1 call per task for input extraction
 - Capability Selection: 1 call per task
- **Total:** ~2-4 calls depending on task count
- **Latency:** +500-1000ms per workflow
- **Cost:** +\$0.0002-\$0.0005 per workflow

Next Steps

1. Implement IOResolver class
2. Implement DependencyRefiner class
3. Update decomposition prompts
4. Add comprehensive tests

5. Update validation to check IO compatibility

Version: 1.0

Status:  Design Complete

Ready for Implementation